

File Upload Portal With Progress And Validation

Task Overview

Build a full stack document management portal similar to a basic Google Drive.

Users should be able to upload files, organise them into folders, preview supported formats, search files, rename files, and delete files. Access must be controlled so users can only manage files they are authorised to use.

Must Include

File Upload

- Drag and drop upload
- Upload using a browse button
- Multiple file upload
- Upload progress indicator
- File type validation
- File size validation
- Clear success and error messages

Supported formats can include:

- JPG
- PNG
- PDF
- DOCX
- XLSX

File Preview

- Image preview for JPG and PNG
- PDF preview
- File icon for other formats
- Show filename, file type, file size, and uploaded date

Folder Management

- Create folder
- Nested folders
- Rename folder
- Delete folder
- Move files into folders
- Breadcrumb navigation

Example:

My Files > Projects > Documents

Search And File Actions

- Search files by name
- Sort by filename, file size, or latest upload
- Rename file
- Delete file
- Download file

Permission Based Access

Use at least two roles:

Admin

- View all files
- Upload files
- Rename and delete files
- Manage folders

User

- Upload files
- View permitted files
- Rename and delete their own files

Permissions must be validated in the backend. A user should not be able to delete another user's file through Postman or browser developer tools.

Loading, Empty, And Error States

- Show upload progress
- Show loading skeletons
- Show an empty state when a folder has no files

- Show useful error messages for failed uploads or restricted access

Technology Requirements

Frontend:

- React.js
- Responsive UI

Backend:

- Node.js
- Express.js
- REST APIs

Database:

- PostgreSQL

Store file details such as filename, file type, file size, folder ID, storage path, owner ID, and uploaded date in PostgreSQL.

Do not store large files directly in the database. Store files in a local uploads folder or cloud storage such as AWS S3 or Cloudinary.

Submission Requirements

Submit:

- GitHub repository
- README file
- Database schema
- API documentation or Postman collection
- Screenshots or deployed demo link
- Meaningful Git commits

Expected Outcome

The final application should work like a lightweight file management system with secure uploads, folder organisation, previews, search, and permission based access.